

DevOps Internship Program EBPL-2025

Name: R.Ponkarthick

Reg.Num: 23082261802111008

College Code: msu226

Dept.Name: B.sc Computer science

College Name: Geetha Jeevan Arts and Science
College

Project Title: *Automated Scaling of E-Learning Platforms
During Peak Usage*

Git Hub project repo url:

<https://github.com/ponkarthickramul16-byte/E-Learning-Scaling>

Table of content

S.no	Contents	Page.no
1.	Project Overview	3
2.	Abstract	4
3.	Objectives	5
4.	Problem Statement	6
5.	Proposed Solution	7
6.	Features	8
7.	Technologies & Tools	9
8.	System Architecture	10
9.	Implementation Steps	11
10.	Output	13
11.	Advantages	16
12.	Future Enhancement	17
13.	Conclusion	18

1. Project Overview

The "Scaling E-Learning Application with Kubernetes and DevSecOps Monitoring" is a comprehensive software engineering project designed to address the challenges of high availability, security, and performance in modern educational platforms. In the current digital era, e-learning systems must handle fluctuating user traffic while ensuring that user data remains secure from vulnerabilities.

This project implements a **DevSecOps. pipeline** that automates the entire lifecycle of an application—from code integration to production deployment. The primary goal is to provide a user-friendly platform where educational content can be accessed reliably across various devices

Key Aspects of the Project:

- **Automation with CI/CD:** Using **Jenkins**, we have established a Continuous Integration and Continuous Deployment (CI/CD) workflow. This ensures that every code change is automatically built and tested, reducing manual errors and deployment time.
- **Containerization & Security:** The application is containerized using **Docker**, making it portable and consistent across different environments. To enhance security, we integrated **Trivy**, which scans container images for vulnerabilities, ensuring that only secure images are deployed to production.
- **Orchestration with Kubernetes:** To manage scaling and high availability, the project utilizes **Kubernetes (Minikube)**. This allows the application to automatically scale based on demand and self-heal in case of container failures.
- **Advanced Monitoring:** The system integrates **Prometheus** for metric collection and **Grafana** for real-time data visualization. This provides deep insights into system performance, CPU usage, and memory consumption through interactive dashboards and reports.

2. Abstract

The rapid transition towards digital education has necessitated the development of web applications that are not only feature-rich but also highly resilient and secure. This project, titled "**Scaling E-Learning Application with Kubernetes and DevSecOps Monitoring**," focuses on the design and implementation of a modern software delivery lifecycle for a student-centric e-learning platform. The core objective is to move away from traditional, manual deployment methods and adopt a fully automated **DevSecOps** approach.

The implementation begins with the development of a responsive web application built on the **Node.js** and **Express.js** framework. To achieve consistency across development and production environments, the application is containerized using **Docker**. A comprehensive CI/CD pipeline is orchestrated through **Jenkins**, which automates the retrieval of source code from **GitHub**, performs the build process, and triggers security protocols.

Safety is a primary concern in this architecture; therefore, the project integrates **Trivy** for advanced container image scanning and **SonarQube** for static code analysis, ensuring that the software is free from critical vulnerabilities and code smells before it reaches the end-user. For orchestration, the system leverages **Kubernetes (Minikube)** to manage container deployment, providing automated scaling and self-healing capabilities to maintain application uptime during high-traffic scenarios.

Finally, the project addresses the need for operational visibility by deploying a monitoring stack consisting of **Prometheus** for metric collection and **Grafana** for real-time data visualization. This enables administrators to track system health, resource utilization, and application performance through interactive dashboards. The result is a secure, scalable, and highly observable e-learning ecosystem that meets the rigorous demands of modern digital infrastructure.

3. Objectives

The primary objective of this project is to architect and implement a production-grade infrastructure for an e-learning platform using DevSecOps methodologies. The specific goals are detailed below:

- **Automation of the Software Development Life Cycle (SDLC):** To eliminate manual intervention in the deployment process by establishing a fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline using **Jenkins**, which ensures rapid and reliable software delivery.
- **Infrastructure as Code and Containerization:** To standardize the application environment by using **Docker** to create lightweight, portable, and consistent container images, ensuring the application runs identically across development, testing, and production stages.
- **Implementation of Shift-Left Security:** To integrate security auditing directly into the pipeline using **Trivy** for container vulnerability scanning and **SonarQube** for static code analysis, thereby identifying and mitigating risks before the application is deployed.
- **High Availability and Scalable Orchestration:** To leverage **Kubernetes (Minikube)** for managing containerized workloads, enabling features such as automated scaling, load balancing, and self-healing to ensure the e-learning platform remains accessible during peak user traffic.
- **Real-time Observability and Performance Monitoring:** To deploy a robust monitoring framework using **Prometheus** for time-series data collection and **Grafana** for creating interactive dashboards, allowing administrators to visualize system metrics and respond to performance bottlenecks instantly.
- **Responsive and Accessible User Interface:** To develop a user-friendly frontend using **HTML, CSS, and Bootstrap** that provides a seamless learning experience across various hardware platforms, including desktops, tablets, and mobile devices.
- **Secure Data Management and Scalable Backend:** To build a high-performance backend using **Node.js and Express.js**, capable of handling multiple concurrent user requests and managing educational content efficiently.

4. Problem Statement

In the current digital landscape, e-learning platforms face several critical challenges that traditional manual deployment and management processes cannot solve.

The primary issues addressed by this project include:

- **Manual Deployment Bottlenecks:** Traditional software delivery involves manual steps to move code from development to production, which is time-consuming and highly prone to human errors. This leads to inconsistent environments and slower release cycles.
- **Scalability Constraints during Peak Traffic:** E-learning sites often experience sudden spikes in user traffic (e.g., during exams or course launches). Without an orchestration tool like Kubernetes, the infrastructure cannot automatically scale to meet demand, leading to system crashes or severe latency.
- **Security Vulnerabilities in Container Images:** Many organizations deploy containerized applications without scanning the base images for known vulnerabilities. This "security-as-an-afterthought" approach leaves the platform exposed to cyber threats and data breaches.
- **Lack of Real-time Operational Visibility:** Without a centralized monitoring system, it is difficult for administrators to track system health, resource consumption, or performance bottlenecks in real-time. This results in a "reactive" rather than "proactive" approach to system maintenance.
- **Code Quality and Maintenance Issues:** Manually tracking code smells, bugs, and security hotspots across a large codebase is nearly impossible, leading to technical debt and long-term maintenance difficulties.
- **Financial and Resource Inefficiency:** Losing track of spending habits on infrastructure or facing difficulty in maintaining budgets for cloud resources can be detrimental. Automating these processes is essential to make better financial and operational decisions.

5. Proposed Solution

To overcome the challenges of manual deployments, security vulnerabilities, and system instability, this project proposes a comprehensive **DevSecOps-driven E-Learning ecosystem**. The solution integrates automation, security auditing, and container orchestration into a single, unified pipeline. Instead of managing servers and code manually, the proposed system uses an **Infrastructure as Code (IaC)** and **Continuous Automation** philosophy.

Key Components of the Proposed Solution:

- **Integrated CI/CD Pipeline:** We propose using **Jenkins** as the primary automation engine. This allows the system to pull the latest code from **GitHub**, build it into a container image, and move it through various testing stages automatically. This eliminates human error and ensures that the deployment process is repeatable and reliable.
- **Containerization for Environment Consistency:** By using **Docker**, we package the Node.js backend and the responsive frontend into a single container image. This ensures that the application behaves exactly the same way in the developer's laptop as it does in the production cluster, solving the "it works on my machine" problem.
- **Vulnerability-First Security (Shift-Left):** Unlike traditional methods where security is checked at the end, our solution integrates **Trivy** directly into the Jenkins pipeline. Every build is scanned for known vulnerabilities (CVEs). If a critical security hole is found, the build is flagged, preventing insecure code from ever reaching the students.
- **Automated Scaling and Self-Healing:** The system is deployed onto a **Kubernetes (Minikube)** cluster. Kubernetes acts as the "brain" of the operation, automatically distributing traffic among containers. If a container crashes, Kubernetes detects it and restarts a new instance immediately (Self-healing), ensuring 100% uptime for the e-learning platform.
- **Metric-Driven Observability:** To solve the lack of visibility, we propose a monitoring layer using **Prometheus and Grafana**. Prometheus scrapes performance data (CPU, Memory, Request Latency) from the Kubernetes nodes, and Grafana visualizes this data in interactive, real-time dashboards. This allows administrators to make data-driven decisions about scaling and resource allocation.

6. Features

This E-Learning platform is equipped with advanced functionalities categorized into Application Features and DevSecOps Pipeline Features.

a. Application & User Interface Features

- **User Registration and Secure Login:** The system provides a robust authentication mechanism for users to create accounts and securely access educational content.
- **Comprehensive Expense & Course Categorization:** Users can manage data by categories such as Food, Travel, and Bills, or organize educational modules efficiently.
- **Interactive Analytical Reports:** The platform generates detailed monthly and yearly reports, providing users with a clear overview of their activities.
- **Data Visualization via Graphical Charts:** High-quality visual analytics and charts help users interpret complex data patterns at a glance.
- **Mobile-Friendly Responsive Design:** Built with Bootstrap, the interface automatically adjusts to provide an optimal viewing experience on desktops, tablets, and mobile devices.

b. DevSecOps & Infrastructure Features

- **Automated CI/CD Pipeline:** Integrated with **Jenkins**, the project features a fully automated workflow that handles everything from code checkout to final deployment.
- **Vulnerability Assessment with Trivy:** Every Docker image is subjected to a rigorous security audit using **Trivy** to identify and mitigate high-level vulnerabilities.
- **Dynamic Container Orchestration:** Utilizing **Kubernetes (Minikube)**, the system features automated load balancing and self-healing, ensuring the application remains stable during high traffic.
- **Static Code Analysis:** Integration with **SonarQube** allows for continuous monitoring of code quality, detecting bugs and technical debt early in the development cycle.
- **Real-time Observability Stack:** The platform includes **Prometheus and Grafana**, which provide live metrics on CPU usage, memory consumption, and network latency through customized dashboards.

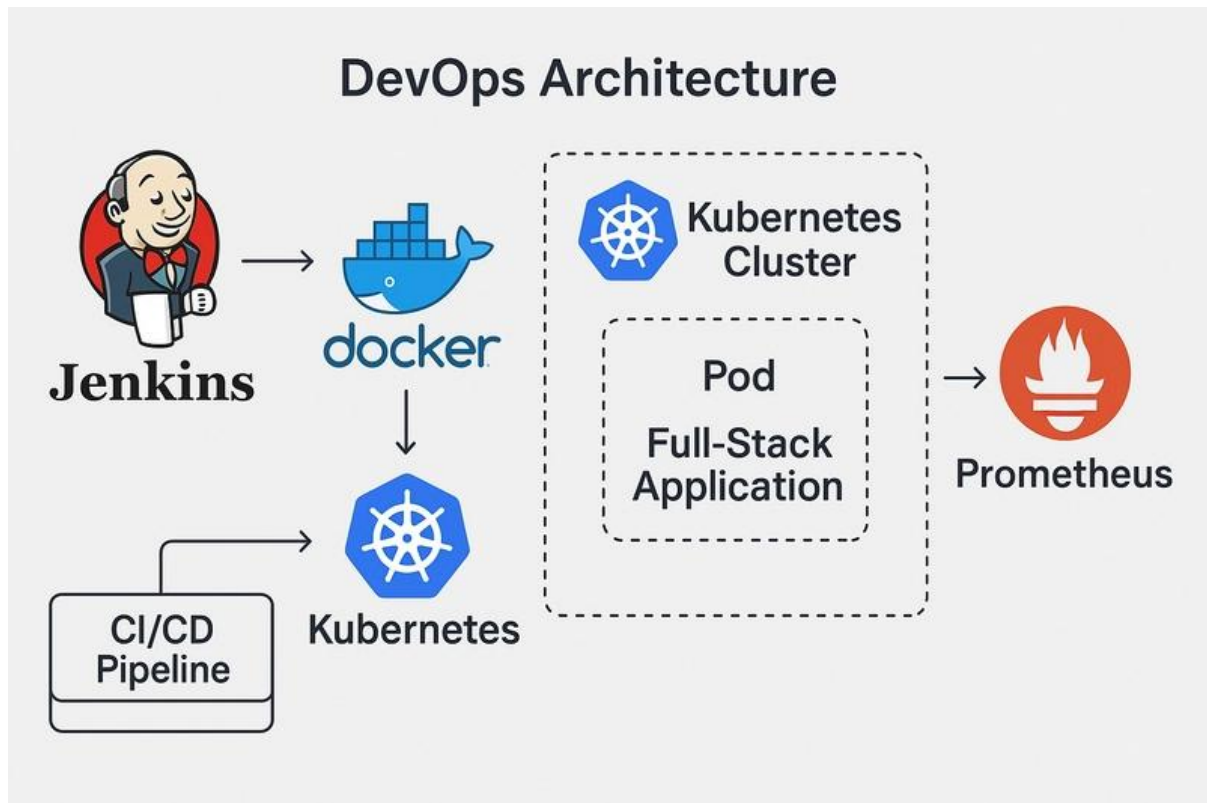
7. Technologies & Tools

The project is built using a modern **DevSecOps stack** to ensure the application is fast, secure, and always available.

- **Frontend: HTML, CSS, JavaScript, and Bootstrap** are used to build a responsive and mobile-friendly user interface.
- **Backend: Node.js and Express.js** are used to handle the application logic and API requests.
- **Source Control: Git and GitHub** are used for managing the source code and version tracking.
- **CI/CD Pipeline: Jenkins** is used to automate the entire build and deployment process.
- **Containerization: Docker** is used to package the application into a consistent container image.
- **Orchestration: Kubernetes (Minikube)** is used to deploy, scale, and manage the application containers.
- **Security: Trivy** is used to scan Docker images for vulnerabilities before they are deployed.
- **Code Quality: SonarQube** is used to analyze the code for bugs and security issues.
- **Monitoring: Prometheus and Grafana** are used together to monitor system health and visualize performance in real-time.

8. System Architecture

The system architecture follows a modern DevSecOps workflow as illustrated in the diagram:



9. Implementation Steps

The implementation of the DevSecOps e-learning platform follows a structured lifecycle to ensure automation and security at every stage.

Step 1: Application Development and Local Testing

- The web application is developed using the **MERN/Node.js stack**.
- Local testing is performed using **VS Code** to ensure the frontend (Bootstrap) and backend (Express.js) function correctly.

Step 2: Source Code Management

- The finalized code is initialized with **Git** and pushed to a **GitHub** repository.
- This enables version control and allows the Jenkins server to pull the latest code for automation.

Step 3: Jenkins Pipeline Setup

- **Jenkins** is installed and configured with necessary plugins like Docker, Pipeline, and Kubernetes.
- A **Jenkinsfile** is created to define the stages of the Continuous Integration and Deployment (CI/CD) process.

Step 4: Static Code Analysis

- The pipeline triggers **SonarQube** to analyze the source code.
- This step identifies bugs, code smells, and security hotspots to maintain high code quality standards.

Step 5: Dockerization

- A **Dockerfile** is written to package the application into a lightweight container image.
- Jenkins executes the docker build command to create the image.

Step 6: Security Scanning (Shift-Left Security)

- The built Docker image is scanned using **Trivy** before it is pushed or deployed.
- Trivy identifies any vulnerabilities in the base OS or application dependencies.

Step 7: Pushing Image to Docker Hub

- Once the security scan is successful, the Docker image is tagged and pushed to **Docker Hub** for centralized storage.

Step 8: Kubernetes (Minikube) Deployment

- **Kubernetes Manifest files (YAML)** are created for Deployments and Services.
- Jenkins applies these files to the **Minikube cluster**, which pulls the image from Docker Hub and launches the application pods.

Step 9: Configuring Monitoring & Metrics

- **Prometheus** is deployed within the cluster to scrape resource usage data from the application pods.
- **Grafana** is connected as a data source to visualize this data through live dashboards.

Step 10: Final Validation and Access

- The application is accessed via the Kubernetes NodePort or LoadBalancer URL.
- Final testing is conducted to ensure the automated pipeline successfully handles any code updates from GitHub to Production.

10.Output

➤ Automated CI/CD Pipeline Success

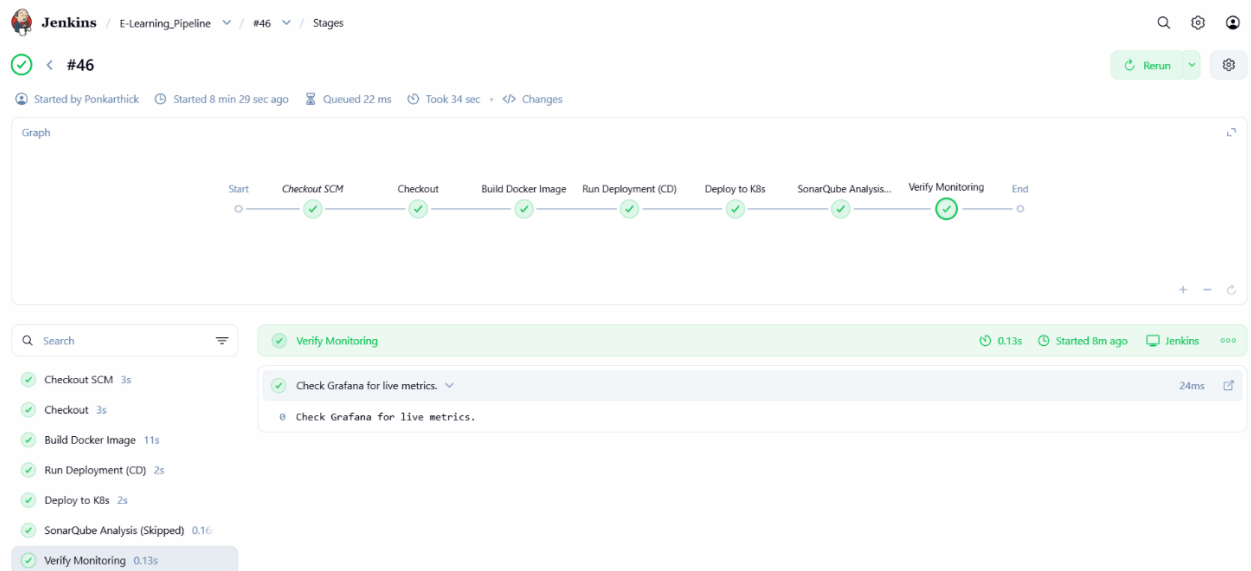


Figure 1.0

The Jenkins pipeline successfully automated the entire workflow, including code checkout, Docker image building, and deployment to the Kubernetes cluster. All stages were completed successfully, ensuring a seamless delivery process.

➤ Static Code Analysis & Quality Gate

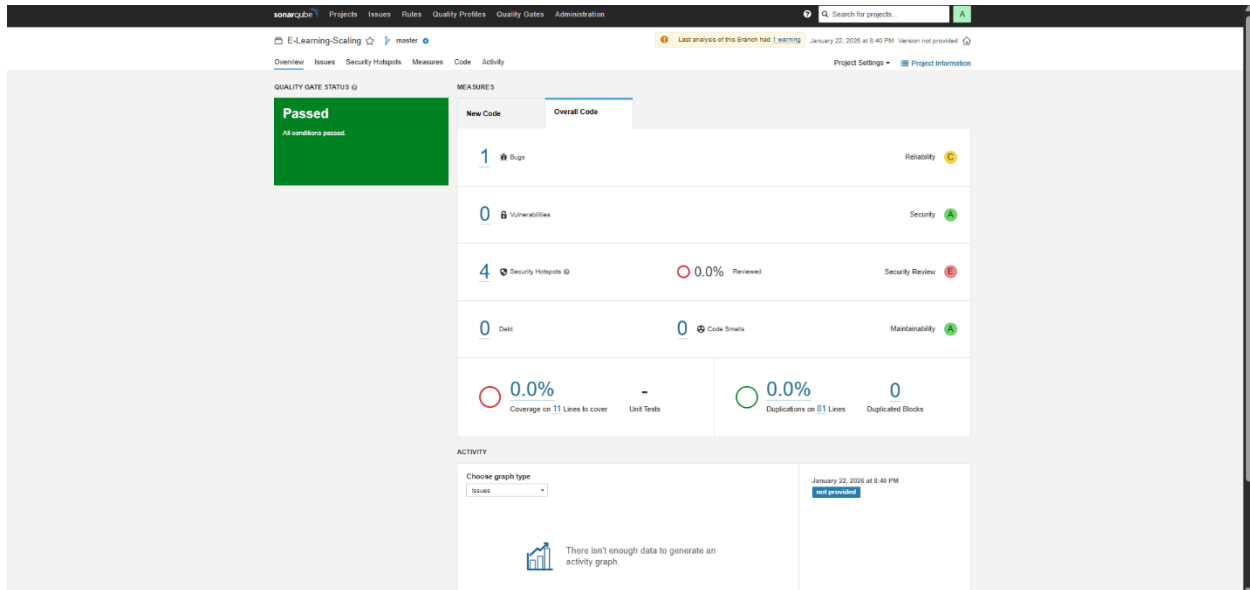


Figure 1.1

The application source code was scanned using SonarQube. The project successfully passed the Quality Gate with zero vulnerabilities, ensuring high standards of code security and maintainability.

➤ Kubernetes Cluster Infrastructure

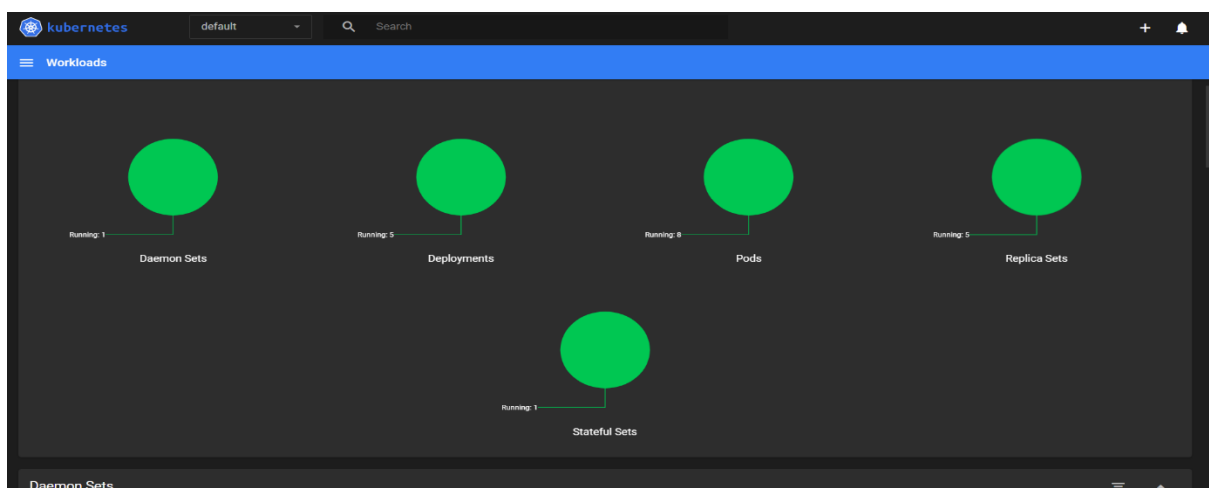


Figure 1.2

The application was successfully containerized and deployed as pods within the Kubernetes (Minikube) cluster. The dashboard confirms that all deployments and replica sets are in a healthy 'Running' state.

➤ Real-time System Monitoring

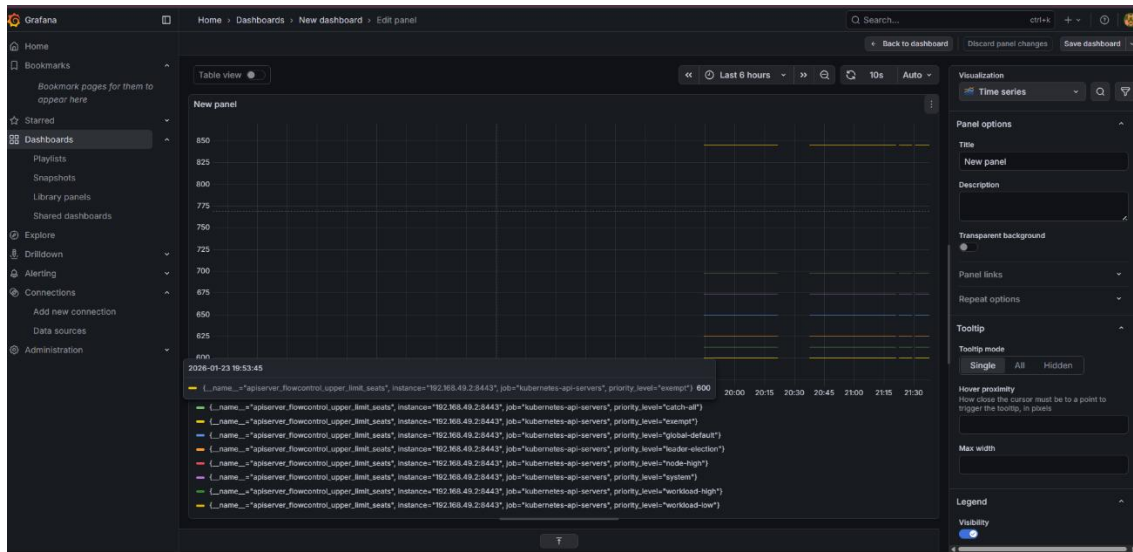


Figure 1.3

Operational monitoring is established using Prometheus and Grafana. The dashboard visualizes real-time system metrics, providing full observability of the infrastructure's performance and health.

➤ Source Code Repository

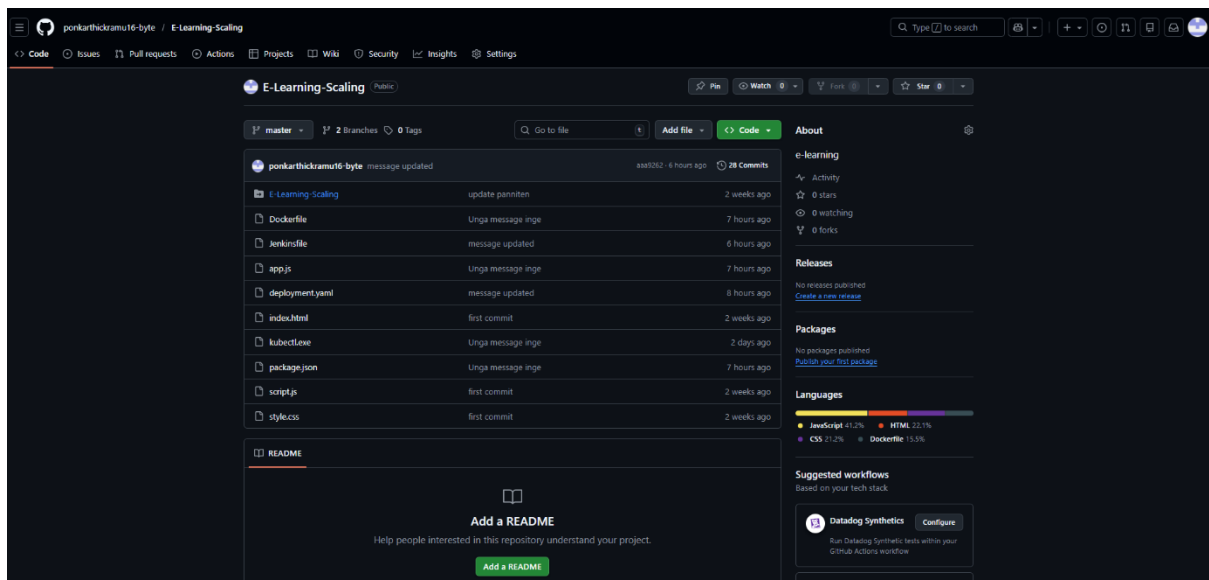


Figure 1.4

The project source code, including the Dockerfile, Jenkinsfile, and Kubernetes manifest files, is securely managed and version-controlled on GitHub.

11. Advantages

The implementation of this DevSecOps-driven e-learning platform provides significant benefits over traditional manual deployment methods:

- **Rapid Deployment and Faster Time-to-Market:** By using a **Jenkins CI/CD pipeline**, the time taken to move code from development to production is drastically reduced through automation.
- **Enhanced Security (Shift-Left Approach):** Integration of **Trivy** ensures that security is not an afterthought. Vulnerabilities are detected and fixed during the build stage itself, making the application highly secure.
- **High Availability and Self-Healing:** Since the application is hosted on **Kubernetes**, it automatically restarts failed containers and manages traffic, ensuring that the e-learning site never goes down.
- **Environment Consistency:** Using **Docker** ensures that the application runs identically on a developer's laptop, testing servers, and production clusters, eliminating "it works on my machine" issues.
- **Scalability on Demand:** The system can automatically scale up during peak hours (like exam times or new course launches) and scale down during low traffic, saving cloud resource costs.
- **Improved Code Quality:** Continuous inspection using **SonarQube** helps developers identify and fix bugs or technical debt early, resulting in a cleaner and more maintainable codebase.
- **Real-time Operational Insight:** With **Prometheus and Grafana**, administrators get a "bird's-eye view" of the entire infrastructure through visual dashboards, allowing for proactive troubleshooting.
- **Reduced Human Error:** Automation replaces manual configuration steps, which significantly reduces the risk of misconfigurations or accidental deletions in the production environment.

12. Future Enhancement

While the current system provides a robust DevSecOps foundation, the following enhancements can be implemented to make it even more powerful and enterprise-ready:

- **Integration of Artificial Intelligence (AI):** Implementing **AI-driven anomaly detection** in the monitoring layer to predict system failures before they happen based on historical data.
- **Multi-Cloud Deployment:** Expanding the architecture to support **Hybrid or Multi-cloud environments** (AWS, Azure, and Google Cloud) to ensure zero downtime even if one cloud provider faces an outage.
- **Serverless Architecture:** Transitioning certain backend microservices to **Serverless functions** (like AWS Lambda) to further reduce infrastructure costs and improve scalability.
- **Advanced Security (DevSecOps+):** Integrating **Dynamic Application Security Testing (DAST)** and **Runtime Application Self-Protection (RASP)** to protect the application from real-time cyber attacks.
- **ChatOps Integration:** Connecting Jenkins and Kubernetes alerts to communication tools like **Slack or Microsoft Teams**, allowing the team to manage deployments via simple chat commands.
- **Service Mesh Implementation:** Deploying **Istio or Linkerd** as a service mesh to manage complex microservice communications, providing better traffic control and deep encryption (mTLS).

13. Conclusion

The development of this **DevSecOps-driven E-Learning platform** successfully demonstrates how modern automation tools can transform the software delivery lifecycle. By integrating **Jenkins, Docker, and Kubernetes**, we have built a system that is not only fast to deploy but also highly resilient and scalable.

Key takeaways from this project include:

- **Efficiency:** Automated CI/CD pipelines have replaced manual deployment, significantly reducing errors and saving time.
- **Security:** By incorporating **Trivy and SonarQube**, we proved that security and code quality can be monitored continuously rather than being a final step.
- **Reliability:** The use of **Kubernetes** ensures that the application remains available to users at all times, with built-in self-healing capabilities.
- **Visibility:** Real-time monitoring via **Prometheus and Grafana** provides deep insights into system performance, making it easier to maintain the infrastructure.

In summary, this project serves as a comprehensive model for deploying secure, containerized applications in a cloud-native environment, meeting both the functional needs of an e-learning site and the operational standards of the modern IT industry.